

Laboratorio 03 ARSW: Immortals & Synchronization

María Paula Rodríguez Muñoz

Juan Andrés Suárez

Juan Pablo Nieto

Tomás Felipe Ramírez

1. Objetivo

El objetivo de este laboratorio es analizar, diseñar y corregir una simulación concurrente de inmortales, identificando y solucionando problemas de condiciones de carrera, sincronización incorrecta, suspensión cooperativa y deadlocks, utilizando Java 21 y una interfaz gráfica en Swing.

Los objetivos específicos incluyen:

- Identificar y corregir condiciones de carrera en las regiones críticas de la simulación.
- Implementar una pausa cooperativa correcta usando mecanismos de sincronización.
- Garantizar el cumplimiento del invariante de la suma total de la salud de los inmortales.
- Analizar y corregir posibles deadlocks mediante una estrategia de sincronización adecuada.
- Validar el comportamiento del sistema con diferentes cantidades de inmortales.

2. Contexto del Problema

La simulación presenta las siguientes características:

- Cada **inmortal** se ejecuta de manera concurrente (un hilo por inmortal).
- Cada inmortal ataca a otro, restándole una cantidad fija de vida y sumando una fracción a su propia vida.
- Todos los inmortales comparten estructuras de datos que representan el estado global de la simulación.
- El sistema permite **iniciar**, **pausar**, **reanudar** y **detener** la simulación.
- Existe un **invariante**: la suma total de la salud de todos los inmortales debe permanecer constante (salvo durante una actualización en curso).

4.2. Solución Eficiente: Monitores (Modo Monitor)

Para solucionar el problema de la espera activa, se implementó una versión eficiente de la cola mediante la clase `EfficientQueue`, apoyándose en el uso de `BoundedBuffer` para manejar la sincronización con monitores.

Mejoras realizadas:

- Uso de `synchronized` para proteger el acceso a los datos compartidos.
- Uso de `wait()`: los hilos liberan el CPU y entran se "duermen" hasta que haya trabajo.
- Uso de `notifyAll()`: los hilos son despertados únicamente cuando el estado de la cola cambia y pueden reanudar su trabajo.

Resultados:

- El consumo de CPU disminuyó a niveles cercanos al 0 %.
- Los hilos permanecen la mayor parte del tiempo en estado `WAITING`, siendo "despertados" solo cuando la cola cambia de estado.

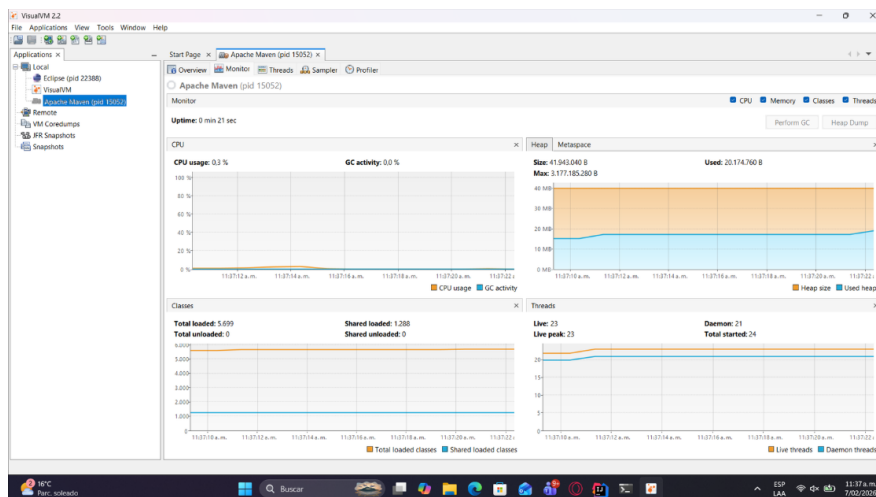


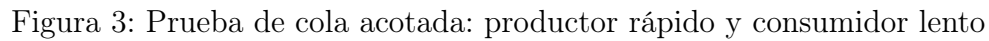
Figura 2: Bajo consumo de CPU usando monitores con `wait()` y `notifyAll()`

4.3. Pruebas de Cola Acotada

Finalmente, se validó el límite de stock con un productor rápido y un consumidor lento. En este escenario se observó que:

- El productor se detiene correctamente al alcanzar la capacidad máxima (`capacity`) de la cola.

- Las Figuras 3 y 5 muestran evidencia de estas pruebas.



Se reescribió el buscador de listas negras para que la búsqueda se detenga tan pronto el conjunto de hilos detecte el número de ocurrencias definido por `BLACK_LIST_ALARM_COUNT`.

Finalización anticipada

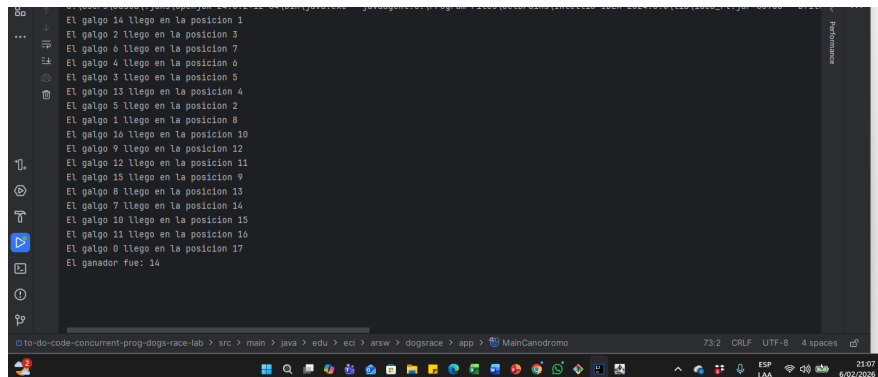
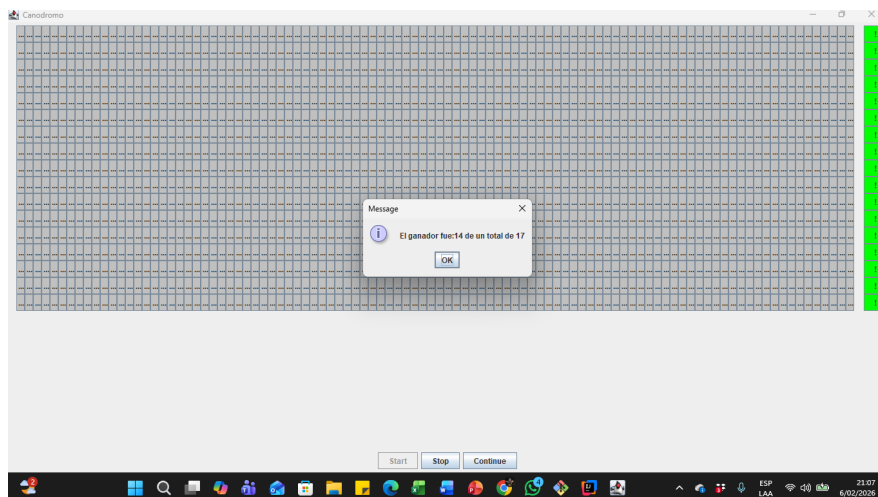
Se implementó una condición de parada global que permite que todos los hilos detengan su ejecución cuando el contador alcanza el umbral establecido. De esta manera, no se recorren los servidores restantes y el método retorna inmediatamente el resultado.

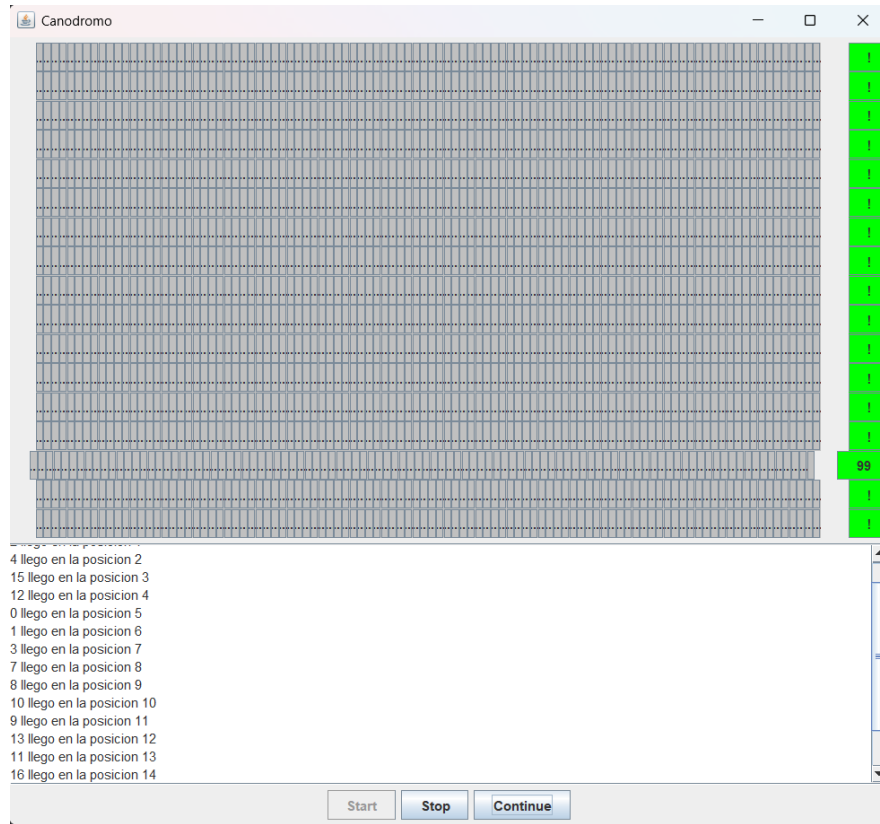
Control de condiciones de carrera

Para garantizar ausencia de condiciones de carrera sobre el contador compartido, se utilizó:

- `AtomicInteger` para realizar incrementos atómicos, o
- Sincronización mínima sobre la región crítica donde se incrementa el contador.

Con esto se asegura consistencia en el valor del contador aun cuando múltiples hilos lo actualicen simultáneamente.





6. Parte III —Sincronización y Deadlocks con Highlander Simulator

6.1. Descripción de la simulación

Se revisa la simulación con N inmortales, donde cada uno ataca a otro. El atacante resta M puntos de vida al contrincante y suma $M/2$ a su propia vida.

6.2. Invariante del sistema

Con N inmortales y salud inicial H , la suma total de la salud debería permanecer constante (salvo durante un update). El valor del invariante se calcula como:

$$\text{Salud total inicial} = N \times H$$

Este valor se utiliza para validar la consistencia de la simulación durante la ejecución. En este caso la suma total de la salud de todos los inmortales no es constante durante la simulación, ya que con cada ataque se pierde $M/2$ de la salud total del sistema. El invariante clave es que

la suma total de la salud debe permanecer constante mientras la simulación está en pausa.

6.3. Prueba con “Pause & Check”

Se ejecuta la interfaz gráfica y se prueba la opción “Pause & Check”. Se verifica si el invariante se cumple al pausar la simulación y se explica el resultado observado.

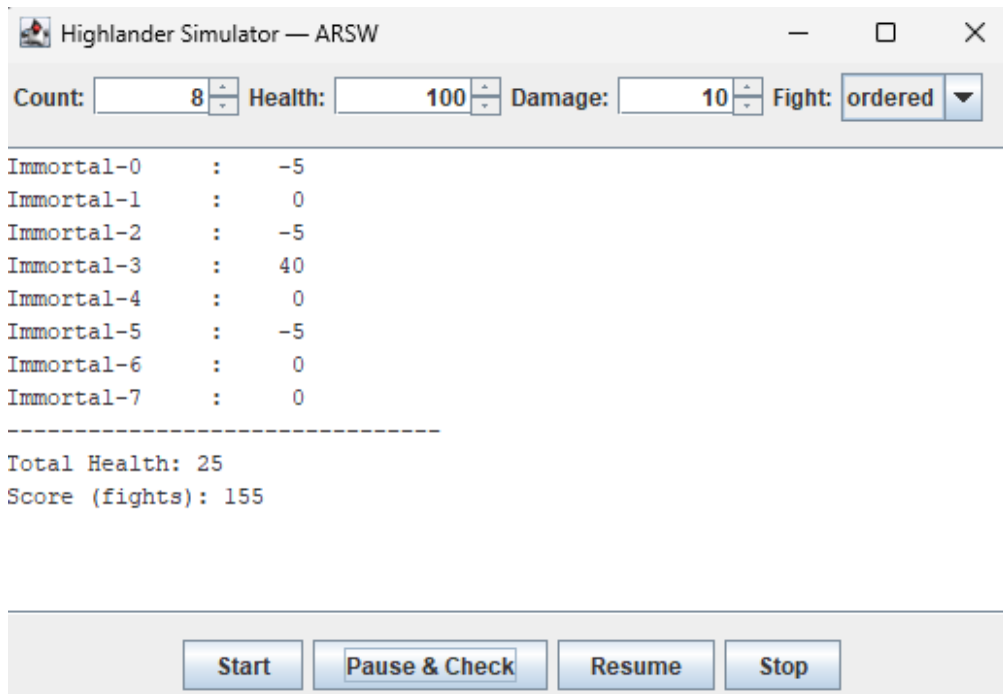


Figura 5: Ejecución de la simulación usando la opción “Pause & Check” con N=8.

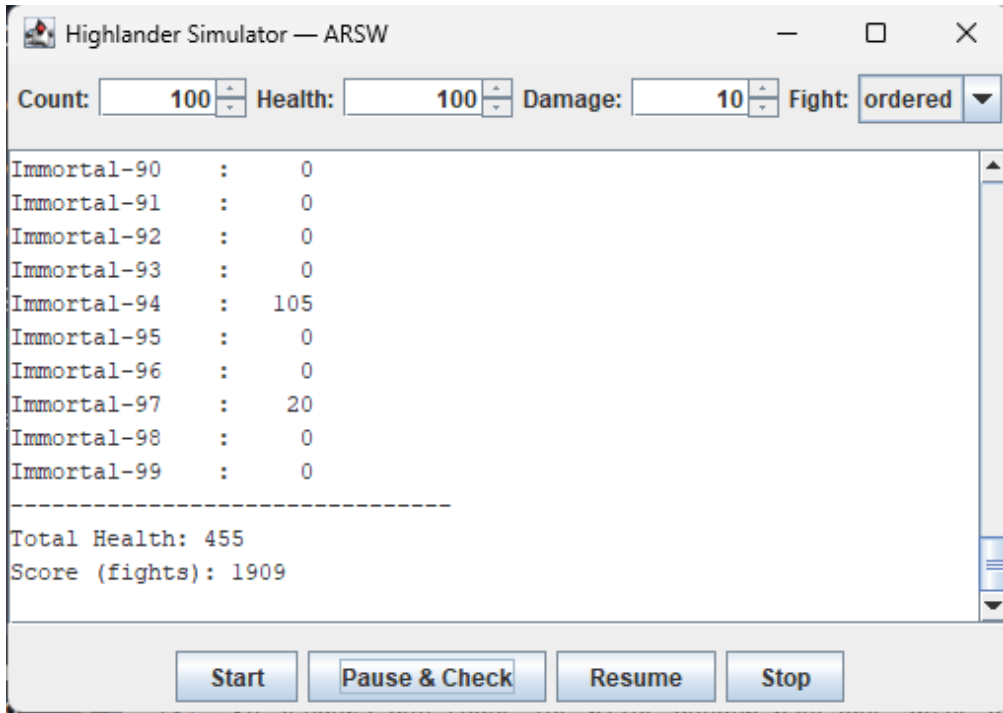


Figura 6: Ejecución de la simulación usando la opción “Pause & Check” con $N=100$.

En este caso, el invariante dice que la suma total de la vida de todos los inmortales debería ser $N \times H$, pero al oprimir “Pause & Check” se puede ver que la suma total no coincide con ese valor. Esto pasa porque en cada pelea un inmortal pierde M puntos de vida y el otro solo gana $M/2$. Si todo estuviera bien sincronizado y las operaciones se hicieran de forma correcta, el total debería mantenerse.

6.4. Implementación de una pausa correcta

Para que la pausa sea efectiva, es necesario que todos los hilos se detengan antes de calcular la salud total. La implementación actual usa un `PauseController`, pero para garantizar la consistencia, se debería esperar un breve momento después de pausar para que las operaciones en curso finalicen. La función de reanudar ya está implementada.

6.5. Validación con múltiples clics

Se hace clic repetidamente en “Pause & Check” y se valida la consistencia de los resultados. Se analiza si el invariante se mantiene en cada verificación.

6.6. Regiones críticas y sincronización

Las secciones críticas son los métodos de lucha (`fightNaive` y `fightOrdered`).

- `fightNaive` puede causar *deadlocks* porque los locks se adquieren en un orden inconsistente.
- `fightOrdered` previene los *deadlocks* al adquirir los locks en un orden predefinido y consistente (basado en el nombre del inmorto).

```
private void fightNaive(Immortal other) {  Ⓜ Maria Paula
    synchronized (this) {
        synchronized (other) {
            if (!this.isAlive() || !other.isAlive())
                return;

            int damageValue = ThreadLocalRandom.current().nextInt(this.damage) + 1;
            int effectiveDamage = Math.min(damageValue, other.health);

            other.health -= effectiveDamage;
            this.health += effectiveDamage;
            scoreBoard.recordFight();

            if (other.health <= 0) {
                population.remove(other);
                System.out.println("[DEATH] %s eliminated by %s".formatted(other.name, this.name));
            }
        }
    }
}
```

Figura 7: `fightNaive`

```
private void fightOrdered(Immortal other) {  Ⓜ Maria Paula *
    Immortal first = this.name.compareTo(other.name) < 0 ? this : other;
    Immortal second = this.name.compareTo(other.name) < 0 ? other : this;
    synchronized (first) {
        synchronized (second) {
            if (!this.isAlive() || !other.isAlive())
                return;

            int damageValue = ThreadLocalRandom.current().nextInt(this.damage) + 1;
            int effectiveDamage = Math.min(damageValue, other.health);

            other.health -= effectiveDamage;
            this.health += effectiveDamage / 2;
            scoreBoard.recordFight();

            if (other.health <= 0) {
                population.remove(other);
                System.out.println("[DEATH] %s eliminated by %s".formatted(other.name, this.name));
            }
        }
    }
}
```

Figura 8: `fightOrdered`

6.7. Diagnóstico de deadlocks

Si la aplicación se congela, se puede usar `jps -l` para obtener el ID del proceso y `jstack <PID>` para analizar los hilos.

El comando `jstack` revelará si hay un *deadlock* y qué hilos están involucrados.

```
Found 1 Java-level deadlock:
=====
"Immortal-3":
  waiting to lock monitor 0x000001e9e8c8b8f0 (object 0x00000007125c04a8, a edu.eci.arsw.immortals.Immortal),
  which is held by "Immortal-7"
"Immortal-7":
  waiting to lock monitor 0x000001e9e8c8a9e0 (object 0x00000007125c05e8, a edu.eci.arsw.immortals.Immortal),
  which is held by "Immortal-3"

Java stack information for the threads listed above:
=====
"Immortal-3":
  at edu.eci.arsw.immortals.Immortal.fightNaive(Immortal.java:111)
  - waiting to lock <0x00000007125c04a8> (a edu.eci.arsw.immortals.Immortal)
  - locked <0x00000007125c05e8> (a edu.eci.arsw.immortals.Immortal)
"Immortal-7":
  at edu.eci.arsw.immortals.Immortal.fightNaive(Immortal.java:111)
  - waiting to lock <0x00000007125c05e8> (a edu.eci.arsw.immortals.Immortal)
  - locked <0x00000007125c04a8> (a edu.eci.arsw.immortals.Immortal)
```

6.8. Estrategia para corregir deadlocks

La estrategia principal para corregir *deadlocks* es usar un orden de bloqueo total, como se implementa en `fightOrdered`.

Alternativamente, se podría usar `tryLock` con un tiempo de espera y reintentos para evitar que los hilos se bloqueen indefinidamente.

6.9. Validación con gran número de inmortales

Al probar con un gran número de inmortales (100, 1000, etc.), el modo `ordered` es más robusto.

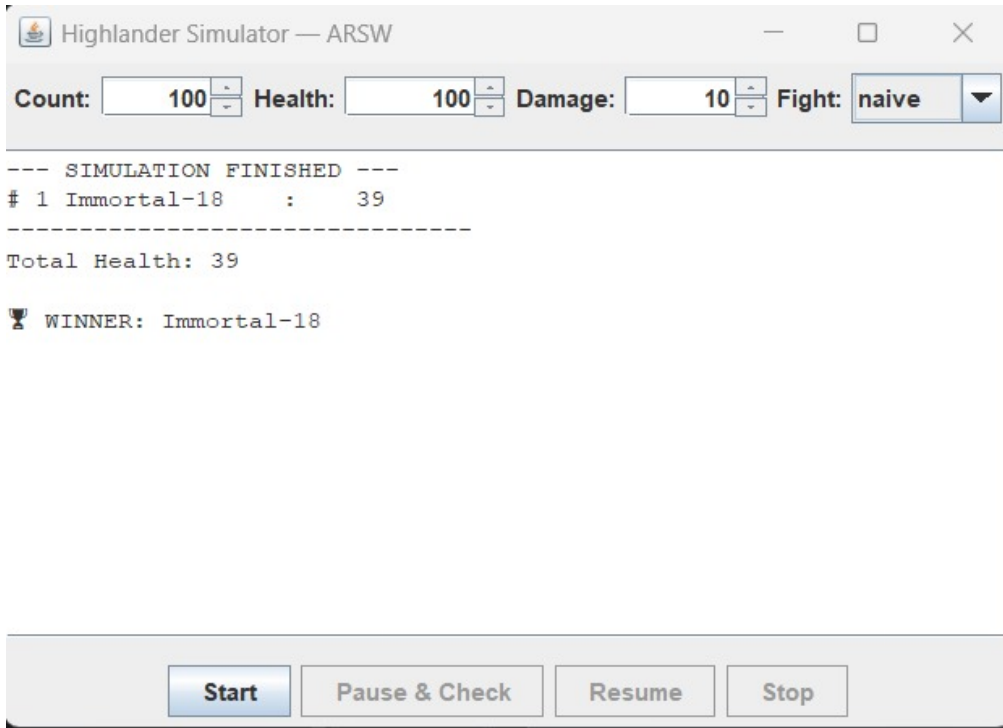


Figura 9: 100 inmortales

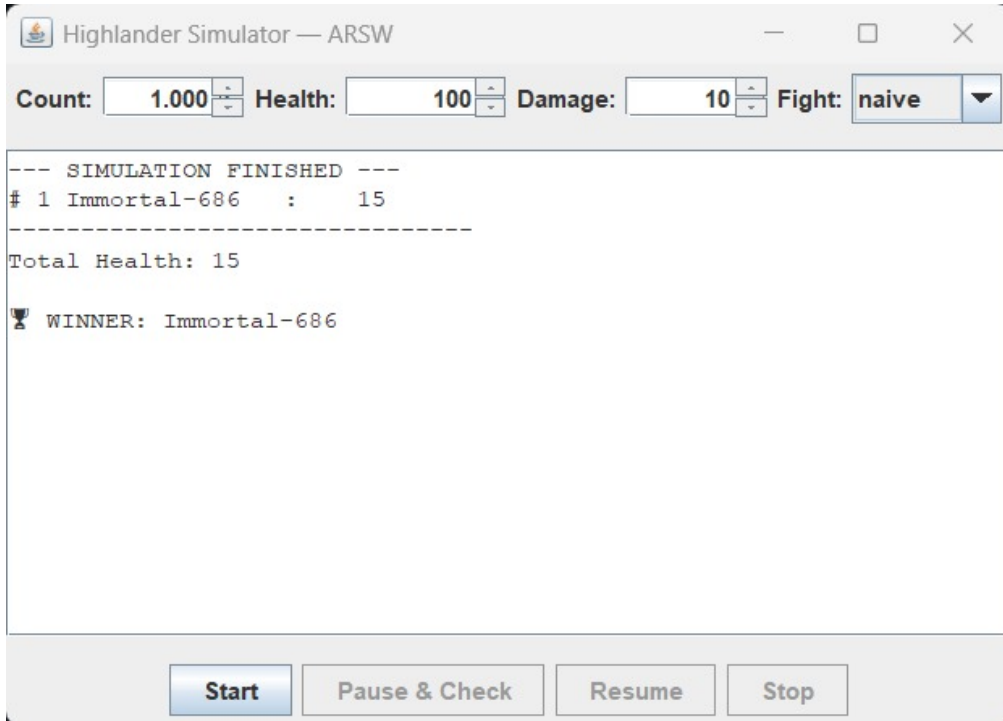


Figura 10: 1000 inmortales

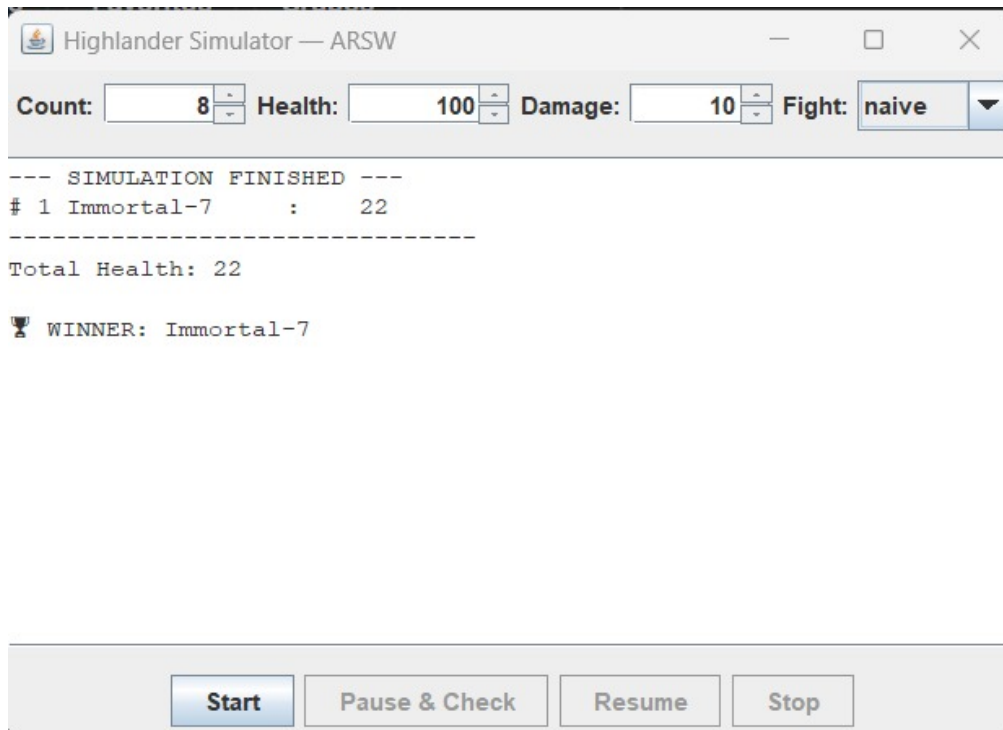
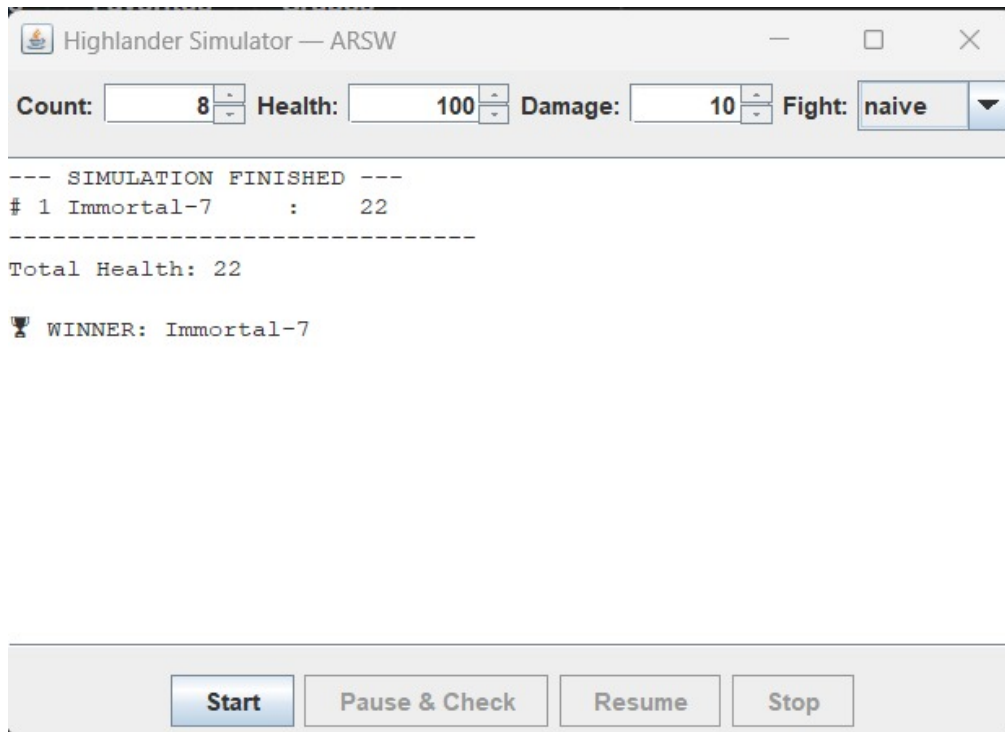


Figura 11: 8 inmortales

6.10. Eliminación de inmortales muertos sin bloquear la simulación

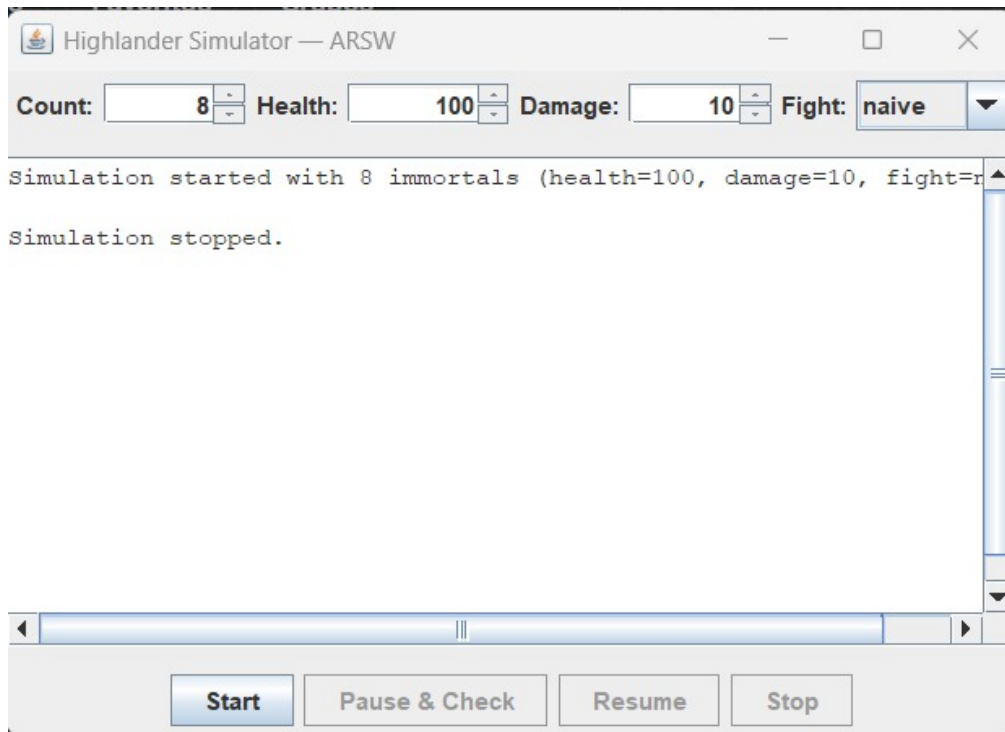
Los inmortales muertos se eliminan de una lista concurrente (`CopyOnWriteArrayList`), lo que evita la necesidad de un bloqueo global. Esta estructura de datos es segura para subprocesos y previene condiciones de carrera durante la eliminación, aunque puede tener un costo de rendimiento si las eliminaciones son muy frecuentes.



6.11. Implementación de STOP (apagado ordenado)

Se ha implementado un método `stop()` que detiene la simulación de forma ordenada.

Este método detiene a todos los inmortales, apaga el `ExecutorService` para interrumpir los hilos y limpia los recursos, asegurando un cierre limpio de la aplicación.



7. Conclusiones

1. La identificación de regiones críticas es fundamental para evitar condiciones de carrera en aplicaciones concurrentes.
2. El uso adecuado de mecanismos de sincronización permite prevenir deadlocks y garantizar consistencia.
3. La pausa cooperativa con `wait()` y `notifyAll()` asegura estados coherentes al verificar invariantes.
4. La validación mediante invariantes es una herramienta clave para detectar errores de sincronización.
5. Un diseño correcto permite escalar la simulación a un gran número de hilos sin perder estabilidad.